Informe del Trabajo Práctico AyED 2026

Profesor: Pablo Mendez
Comisión: K1103

Legajo	Apellido Y Nombre	Correos de los integrantes
215.164-9	Reversat Bryan Alberto	breversat@frba.utn.edu.ar
233.669.8	Aliaga Choque Santiago Luis	saliagachoque@frba.utn.edu.ar
233.837.3	Di Vito, Nicolás Gabriel	nidivito@frba.utn.edu.ar

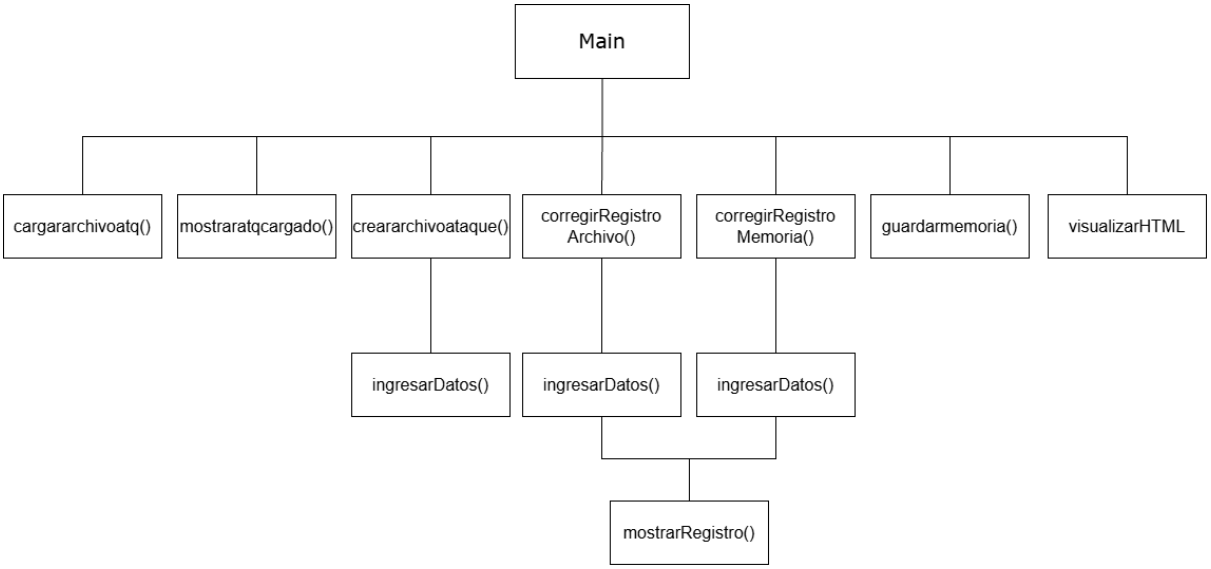
Comenzando el trabajo práctico lo primero que hicimos fue escribir las estructuras dadas en el enunciado para poder crear el programa principal, donde se encuentra el menú del sistema, que nos lleva a cada una de las operaciones.
Para realizar el código nos dividimos las tareas por operaciones, donde nos organizamos de la siguiente manera:
Di Vito, Nicolás: Operación 1, 2 y 6.
Bryan: Operación 3, 4 y 7.
Aliaga Choque, Santiago: Programa principal, operación 5.

Supuestos e hipótesis de diseño:

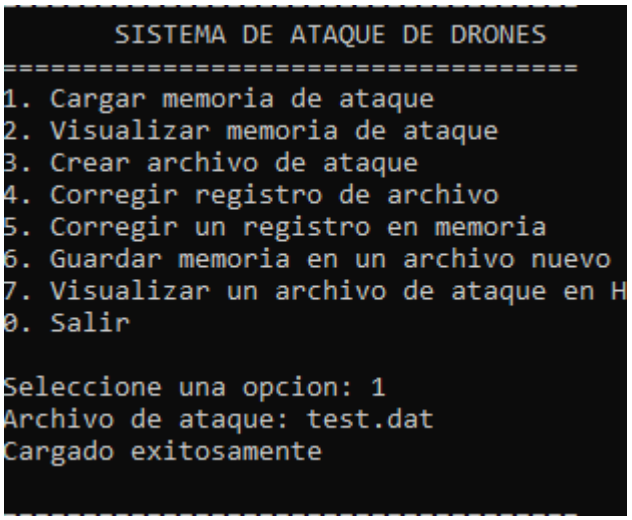
Para el correcto desarrollo y ejecución del sistema de simulación, se establecen una serie de hipótesis y restricciones lógicas que sirven como base para la arquitectura del software:

- **Delimitación espacial:** Se asume que el espacio operativo está acotado estrictamente a una cuadrícula bidimensional fija de 200×200, por lo que cualquier intento de acceder a posiciones fuera de este rango se controla mediante validaciones lógicas para evitar fallos o desbordamientos en la memoria RAM.
- **Persistencia estructurada:** Se establece que el almacenamiento en disco se realiza mediante archivos binarios estructurados en bloques de tamaño fijo (sizeof(OrdenArchivo)), lo cual garantiza la integridad de los registros al utilizar funciones de posicionamiento exacto como tellg() y seekp().
- **Control de estados y errores operativos:** Se contempla que el sistema debe anticiparse a situaciones imprevistas —como el ingreso de nombres de archivos sin extensión o la manipulación de celdas que aún no poseen información cargada— implementando filtros condicionales y estados de control (ocupado) previos a cualquier ejecución de lectura, escritura o actualización en la memoria.

Diagrama de bloques:



OPERACIÓN 1:


SISTEMA DE ATAQUE DE DRONES
1. Cargar memoria de ataque
2. Visualizar memoria de ataque
3. Crear archivo de ataque
4. Corregir registro de archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en H
0. Salir
Seleccione una opcion: 1
Archivo de ataque: test.dat
Cargado exitosamente

OPERACIÓN 2:

```
Seleccione una opcion: 2
___ Inicio del ataque ___
Posicion [2][2]: Despegue. (Siguiente: 2,3)
Posicion [2][3]: Espera 2seg. Granada 1. (Siguiente: 3,3)
Posicion [3][3]: Kamikaze. (Siguiente: 3,4)
___ Fin del Ataque ___
```

OPERACIÓN 3: Crear un archivo de ataque nuevo

Esta operación permite generar un nuevo archivo de ataque con extensión `.dat`, donde se almacenan los registros que conforman la ruta del dron. Solicitamos al usuario que ingrese el nombre del archivo que desea crear. Luego, el programa verifica que el archivo no exista previamente, para evitar sobrescribir información. Si el archivo ya existe, se informa al usuario y se cancela la operación.

Una vez comprobado que el archivo no existe, se abre en modo binario para comenzar a guardar los registros. El usuario indica cuántas órdenes desea cargar y, por cada una, ingresa sus datos: coordenadas X e Y, tiempo de espera, acciones que debe realizar el dron (como soltar granadas, realizar un ataque kamikaze, aterrizar o despegar) y las coordenadas de la siguiente posición.

Cada registro se guarda en el archivo utilizando la función `write()`, que permite almacenar la información en formato binario. Finalmente, se cierra el archivo y se informa que fue creado correctamente.

```
Seleccione una opcion: 3

Ingrese el nombre del archivo: test

Ingrese los datos de la orden:

Coordenada X: 2
Coordenada Y: 2
Tiempo de espera: 0
Soltar granada 1 (1 = Si / 0 = No): 0
Soltar granada 2 (1 = Si / 0 = No): 0
Ataque kamikaze (1 = Si / 0 = No): 0
Aterrizaje (1 = Si / 0 = No): 0
Despegue (1 = Si / 0 = No): 1
Siguiente X: 2
Siguiente Y: 3

Desea ingresar otra orden? (s/n): s

Ingrese los datos de la orden:

Coordenada X: 2
Coordenada Y: 3
Tiempo de espera: 2
Soltar granada 1 (1 = Si / 0 = No): 1
Soltar granada 2 (1 = Si / 0 = No): 0
Ataque kamikaze (1 = Si / 0 = No): 0
Aterrizaje (1 = Si / 0 = No): 0
Despegue (1 = Si / 0 = No): 0
Siguiente X: 3
Siguiente Y: 3
```

OPERACIÓN 4: Corregir un registro de un archivo de ataque.

Esta operación permite modificar un registro específico de un archivo de ataque que ya existe, sin necesidad de crear uno nuevo ni modificar todos sus registros.

Primero, se solicita el nombre del archivo que se desea modificar y las coordenadas X e Y del registro que se quiere corregir.

El programa abre el archivo en modo lectura y escritura binaria. Luego, recorre sus registros uno por uno, comparando las coordenadas almacenadas con las ingresadas por el usuario.

Cuando encuentra el registro correspondiente, solicita nuevamente todos sus datos para reemplazar la información anterior. Para ello, utiliza `tellg()` para obtener la posición del registro dentro del archivo y `seekp()` para volver a esa posición y escribir los nuevos datos mediante `write()`.

De esta manera, se modifica únicamente el registro seleccionado, conservando el resto de la información del archivo. Finalmente, se cierra el archivo y se informa si la modificación se realizó correctamente o si no se encontró el registro solicitado.

```
Seleccione una opcion: 4

Ingrese el nombre del archivo: test.dat

Ingrese la coordenada X del registro: 3
Ingrese la coordenada Y del registro: 3

Registro encontrado.

----- DATOS DE LA ORDEN -----
Coordenada X: 3
Coordenada Y: 3
Tiempo de espera: 0
Granada 1: 0
Granada 2: 1
Ataque kamikaze: 0
Aterrizaje: 0
Despegue: 0
Siguiente X: 3
Siguiente Y: 4
-----------------------------
```

OPERACIÓN 5:

Esta opción sirve para corregir registros directo en la memoria RAM, por si necesitas cambiar un movimiento o una orden en una casilla específica mientras el programa está corriendo.

Para arrancar, se usa una función `void` que le pide al usuario las coordenadas X e Y. El programa se frena con un `if` para revisar que esas coordenadas estén dentro del rango válido de la matriz y que la celda realmente tenga algo cargado (validando el estado de ocupado).

Una vez que pasa esa validación, el sistema copia los datos que están en la estructura `memoria[x][y]` hacia un `struct` auxiliar temporal. Eso permite mostrar los datos actuales por pantalla y pedirle al usuario los nuevos valores.

Por último, esos datos nuevos se vuelven a copiar a la estructura de la memoria principal, pisando a los viejos. Así es como el registro se actualiza al instante en la sesión actual, sin tocar ningún archivo del disco.

```
0. Salir

Seleccione una opcion: 5

Ingrese la coordenada X del registro: 3

Ingrese la coordenada Y del registro: 3

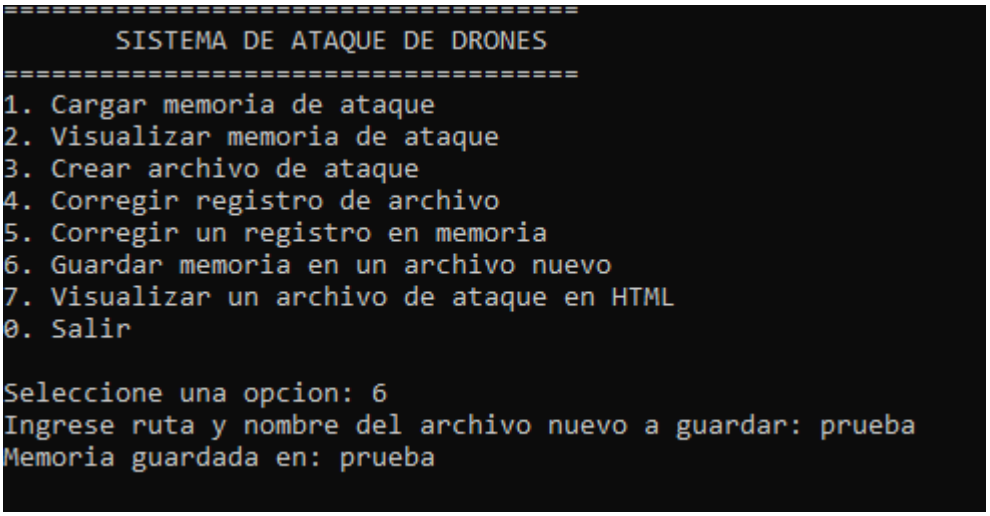
----- DATOS DE LA ORDEN -----
Coordenada X: 3
Coordenada Y: 3
Tiempo de espera: 0
Granada 1: 0
Granada 2: 1
Ataque kamikaze: 0
Aterrizaje: 0
Despegue: 0
Siguiente X: 3
Siguiente Y: 4
-----------------------------

Ingrese los nuevos datos:

Coordenada X: 3
Coordenada Y: 3
Tiempo de espera: 0
Soltar granada 1 (1 = Si / 0 = No): 0
Soltar granada 2 (1 = Si / 0 = No): 0
Ataque kamikaze (1 = Si / 0 = No): 1
Aterrizaje (1 = Si / 0 = No): 0
Despegue (1 = Si / 0 = No): 0
Siguiente X: 3
Siguiente Y: 4

Registro corregido en memoria.
```

OPERACIÓN 6:


SISTEMA DE ATAQUE DE DRONES
1. Cargar memoria de ataque
2. Visualizar memoria de ataque
3. Crear archivo de ataque
4. Corregir registro de archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en HTML
0. Salir
Seleccione una opcion: 6
Ingrese ruta y nombre del archivo nuevo a guardar: prueba
Memoria guardada en: prueba

OPERACIÓN 7: Generar una tabla HTML de los ataques cargados en memoria

Esta operación permite generar un archivo HTML que representa visualmente la matriz de ataques que se encuentra cargada en memoria.

Primero, se solicita al usuario el nombre que desea asignarle al archivo HTML. Si no incluye la extensión `.html`, el programa se encarga de agregarla automáticamente.

Luego, se crea el archivo utilizando la función `fopen()` en modo escritura. Mediante `fprintf()`, se escriben las etiquetas necesarias para construir la estructura del documento HTML, incluyendo la tabla y sus filas y columnas.

El programa recorre la matriz completa de 200x200 posiciones. Para cada posición, comprueba si existe una orden cargada:

- Si la posición está vacía, se muestra un guión (`-`).
- Si contiene una orden, se muestran las acciones correspondientes, el tiempo de espera y las coordenadas de la siguiente posición.

Una vez recorrida toda la matriz, se escriben las etiquetas de cierre del documento y se utiliza `fclose()` para cerrar el archivo.

Finalmente, el usuario puede abrir el archivo HTML generado desde un navegador web y visualizar la matriz de ataques cargada en memoria.

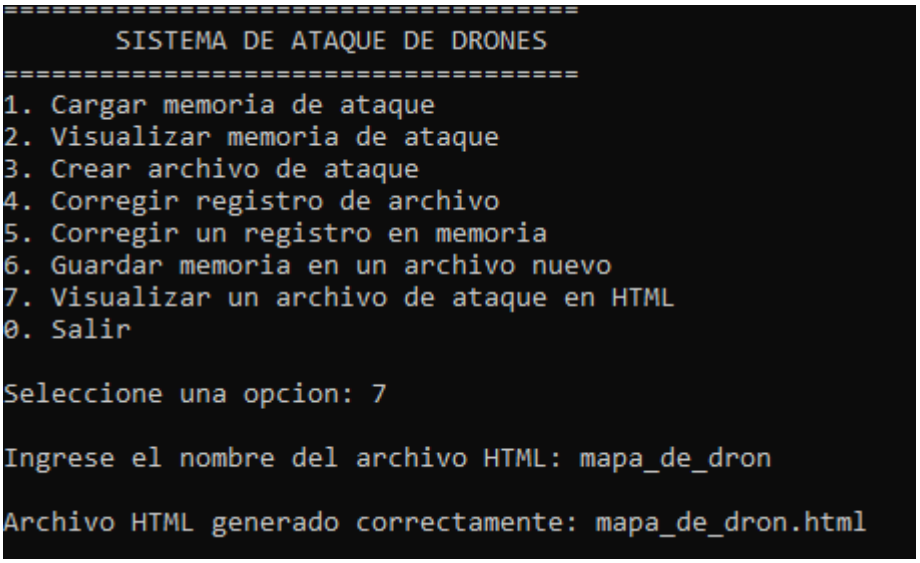


Tabla de ataques del dron

-	-	-
-	-	-
Despegue Sig: (2,3)	Granada 1 Espera: 2 seg Sig: (3,3)	-
-	Granada 2 Sig: (3,4)	Aterrizaje Sig: (3,4)
-	-	-